Wafer Map Tool

Architecture, Pipeline, GUI & Automation

Technical Reference

Component	Wafer Map Tool (`wafer_tool` package)
Scope	EFF ingestion, wafer-map rendering, PDF/PPTX/CSV reporting, interactive GUI, offline automation
Platform	Windows (PyQt6); headless runner is cross-platform
Rendering	Matplotlib (Agg) + SciPy interpolation

Generated July 30, 2026

Contents

Chapter 1

Introduction

The **Wafer Map Tool** turns per-die parametric measurements into colour-coded wafer maps and packages them into a multi-page **PDF**, a **PowerPoint** deck, and a summary **CSV**. It accepts two kinds of input:

- **Raw `.eff` extraction files** — the native semicolon-delimited export from the eSquare / EBS-XE test flow, containing many measurement parameters. The tool scans the header, lets the user pick which parameters to map, and produces one output folder per parameter.
- **Converted `.txt`/`.csv` files** — a single measurement column already extracted to a four-column `lot wafer` / `x` / `y` / `value` layout.

The application has three faces sharing one rendering core:

1. An interactive **PyQt6 GUI** with live wafer-thumbnail preview, threshold scatter and histogram tabs, and a per-parameter preview pager.
2. A **headless runner** that replays a saved job with no GUI, suitable for scheduled / offline execution.
3. A **Windows Task Scheduler** integration that runs saved jobs daily / weekly / monthly and reports their status.

1.1 Reading this document

Chapter 2 gives the module map and layering rules. Chapters 3–7 follow the data through the pipeline, from file formats to the rendered report. Chapter 8 covers the GUI and Chapter 9 the automation. Chapters 10–12 discuss the concurrency model, performance and the modularity assessment.

Chapter 2

High-Level Architecture

2.1 Layered design

The package is organised as a set of layers with a strict downward dependency rule: higher layers may import lower ones, never the reverse. The GUI and the headless runner are two *drivers* that sit on top of the same GUI-free core.

```
+---------------------------------------------------------------+
|  DRIVERS                                                       |
|  ui/ (PyQt6 widgets, main window)   automation/runner (CLI)   |
+------------------------+--------------------------------------+
|  ORCHESTRATION         |                                      |
|  report.generate_report        eff_pipeline.process_eff       |
|  eff_worker (Qt)  worker (Qt)   automation/job, scheduler      |
+---------------------------------------------------------------+
|  EXPORTERS                                                     |
|  exporters/pdf_exporter  pptx_exporter  csv_exporter           |
|  exporters/report_charts (scatter / histogram pages)           |
+---------------------------------------------------------------+
|  CORE RENDERING & DATA                                         |
|  plotting  geometry  statistics  data_loader  eff_loader       |
|  config (PlotConfig, ColorScheme)   logger   exceptions        |
+---------------------------------------------------------------+
             (compiled C parser: eff_parser is external)
```

2.2 Module responsibilities

Module	Responsibility
`config.py`	`PlotConfig` (thresholds, log, mirror, rotation) and `ColorScheme`; tuning constants (`GRID_RESOLUTION`, `MAX_WAFERS_PER_PAGE`, `PAGE_PNG_DPI`).
`geometry.py`	Spatial transforms (mirror / rotation) and `build_wafer_grid` — the SciPy `griddata` interpolation onto a square mesh.
`plotting.py`	`draw_wafer_ax` — renders one wafer contour map onto a Matplotlib axes.
`statistics.py`	8-condition classification counts and the summary bar chart page.

Module	Responsibility
`data_loader.py`	Reads converted `.txt`/`.csv` into a typed DataFrame (`lot, wafer, x, y, value`).
`eff_loader.py`	Self-contained `.eff` reader: header scan, coordinate / measurement-column resolution, spec-limit capture, streaming per-parameter extraction, and an in-memory single-parameter reader for previews.
`report.py`	End-to-end pipeline orchestrator: load → statistics → per-wafer PNGs → PDF → CSV → PPTX.
`eff_pipeline.py`	GUI-free orchestration for the multi-parameter EFF workflow (`process_eff`); shared by the GUI worker and the headless runner.
`eff_worker.py`	Thin Qt `QThread` adapters: `EffReportWorker` and `EffPreviewWorker`.
`worker.py`	`ReportWorker` `QThread` for the single-file (txt/csv) workflow.
`exporters/`	`pdf_exporter`, `pptx_exporter`, `csv_exporter`, and `report_charts` (overview scatter/histogram).
`ui/`	`main_window`, `histogram_widget`, `eff_param_dialog`, `orientation_widget`, `threshold_viz_widget`.
`automation/`	`job` (the `.wtjob` format), `runner` (headless replay), `scheduler` (Task Scheduler), `scheduler_dialog`.

2.3 Key design principle

The most important structural decision is that **all report generation is GUI-free**. `report.generate_report` and `eff_pipeline.process_eff` depend only on Matplotlib/pandas/SciPy, never on PyQt. This is what makes the same logic runnable both interactively (behind a `QThread`) and headless (from the scheduler), guaranteeing identical output.

Chapter 3

Data Model & File Formats

3.1 The EFF extraction file

An `.eff` file is a *semicolon-delimited text* export. After a block of `«Section»` metadata it carries a set of `<...>` header rows and then one row per die:

```
<<EFF:1.00>>;Headers=24;Rows=1527;Columns=109;...
<DataType>;Text;Int;Int;Int;...;Double;Double;...
<+ParameterName>;Lot;Wafer;X;Y;...;RON_VD__10_75;...
<SourceTable>;;;MeasPartPos;MeasPartPos;...;MeasPartVals;...
<LIMIT:SPEC:LOWER_VALUE>;;;;;...;0.0;...
<LIMIT:SPEC:UPPER_VALUE>;;;;;...;31.0;...
1E351778;01;8;5;...;-113.25;...        <- data rows
```

Resolution rules (implemented in `eff_loader.py`, mirroring the standalone `eff_converter`):

- **Coordinates** — the columns named `Lot` (or `LotNumber`), `Wafer`, `X`, `Y` are located case-insensitively and are always loaded.
- **Measurement parameters** — a column is selectable when its `<SourceTable>` is `MeasPartVals`, or (fallback) it is a floating type carrying spec limits.
- **Spec limits** — `<LIMIT:SPEC:LOWER_VALUE>` and `UPPER_VALUE` are captured per parameter and used to pre-fill the GUI thresholds.

The row count (`Rows=`) is read from the `«EFF»` line for a progress estimate; only the header is read during a *scan*, so scanning a multi-GB file is fast.

3.2 The converted TXT layout

The single-parameter text file is four tab-separated columns:

```
1E351778 01<TAB>8<TAB>5<TAB>-113.25
   |          |    |    |
   lot wafer  x    y    value
```

`data_loader.read_wafer_data` parses this with pandas' fast C engine, forcing the identifier column to text (so leading-zero wafer ids like "01" survive) and letting numeric columns parse natively.

3.3 PlotConfig & ColorScheme

`PlotConfig` is the single typed bundle of user plotting choices passed through the whole pipeline:

```
@dataclass
class PlotConfig:
    t_low: float           # Limit 1
    t_high: float          # Limit 2  (auto-ordered t_low <= t_high)
    use_log: bool = False
    high_is_green: bool = False
    mirror_x: bool = False
    mirror_y: bool = False
    rot_deg: int = 0       # one of 0/90/180/270
```

`ColorScheme.from_mode(high_is_green)` yields the three zone colours (low/mid/high or the reverse). Zones: value $<$ Limit 1, Limit 1 $\leq$ value $\leq$ Limit 2, and value $>$ Limit 2.

3.4 The job file (.wtjob)

A `.wtjob` is JSON capturing everything "Generate Report" needs, so a run can be replayed offline. Beyond the `PlotConfig` fields it stores:

Field		Meaning
`input_file`		Fixed input path (txt/csv or `.eff`).
`input_dir` `input_pattern`	+	Folder mode: pick the newest file matching the pattern at run time (e.g. `*.eff`) — so a scheduled job always processes the latest drop.
`out_dir`		Output directory.
`eff_params`		For raw `.eff`: the parameter names to map (one folder each). Empty means "all parameters".

Chapter 4

The Report Pipeline

4.1 `generate_report`

The single-file pipeline is `report.generate_report(filepath, config, out_dir, on_progress, on_wafer_ready, render_individual_pngs, jobs)`. Its stages:

1. **Load** the data (`read_wafer_data`).
2. **Statistics** — `calculate_8_condition_statistics` classifies every die into one of eight conditions.
3. **Per-wafer PNGs** (optional) — one square archive image per wafer, rendered in parallel; feeds the live GUI preview and lands in the output.
4. **PDF** — `generate_pdf` builds the multi-page report.
5. **CSV** — `export_color_summary_csv`.
6. **PPTX** — `create_powerpoint_report` assembles page snapshots.

Two callbacks decouple the pipeline from any UI: `on_progress(current, total, message)` and `on_wafer_ready(lot, wafer, png_path)`. A `RuntimeError` raised from `on_progress` is the cancellation channel.

4.2 Progress & cancellation contract

`total_steps = n_wafers + 3` (three export steps). The wafer-render phase reports one step per wafer; the last three are PDF / CSV / PPTX. The GUI worker translates these into two progress bars and cancels by raising `RuntimeError("Cancelled by user.")` from its progress slot.

Chapter 5

EFF Ingestion Pipeline

The EFF workflow turns *one* file with *many* parameters into one report per selected parameter. It is deliberately split so the interactive and automated paths run identical logic.

5.1 Scan

`eff_loader.scan_eff(path)` reads only the header and returns an `EffScan`: declared row/column counts, the resolved coordinate indices, and a list of `EffParameter` (index, name, dtype, `has_limits`, `limit_lower`, `limit_upper`). It stops at the first data row, so it is fast on very large files.

5.2 Extraction

`extract_parameters(path, indices, out_base_dir, ...)` streams the whole file *once* and writes one four-column text file per selected parameter into its own folder:

```
<out_dir>/<parameter>/<parameter>.txt
```

Invalid values (empty, non-numeric, or $|v| \geq 10^{10}$ sentinels) are skipped. The per-parameter folder subsequently receives that parameter's PNGs, PDF, PPTX and CSV.

5.3 Orchestration: `process_eff`

`eff_pipeline.process_eff` is the GUI-free core:

1. Extract all selected parameters in one streaming pass (~30% of the progress bar).
2. For each parameter that yielded data, call `generate_report` into that parameter's folder.
3. Flatten the per-wafer PNGs up into the folder so images + PDF + PPTX + CSV sit together.

Progress is reported as `(overall_pct, wafer_pct, message)`; cancellation is an `EffCancelled` exception (converted to a `RuntimeError` inside the `generate_report` call so the render aborts cleanly).

5.4 In-memory preview reader

`extract_single_param_df` reads exactly one parameter into a DataFrame without touching disk. The GUI uses it (on a background thread) to draw the scatter/histogram preview for the currentlyselected parameter.

Chapter 6

Rendering Internals

6.1 Interpolation (geometry.build_wafer_grid)

Each wafer's scattered (x, y, value) dies are interpolated onto a GRID_RESOLUTION^2 (default 100×100) square mesh with SciPy griddata, clamped to the data range, then masked to the wafer disc. This is the CPU-heavy step.

6.2 Drawing (plotting.draw_wafer_ax)

Applies the spatial transforms, interpolates, maps values through the log-or-linear threshold levels, and paints three filled contour bands with the ColorScheme colours plus the wafer-edge circle.

6.3 Parallel rendering

Per-wafer PNGs and PDF grid pages are independent, so both render across a ProcessPoolExecutor. On Windows each worker re-imports Matplotlib (~7 s fixed spawn cost), so pools are only used when they pay off:

Pass	**Parallel threshold**
Per-wafer PNGs (report._render_wafer_pngs)	≥ 200 wafers (_PNG_PARALLEL_MIN).
PDF grid pages (pdf_exporter._render_all_grid_pages)	≥ 8 pages ≈ 200 wafers (_PDF_PARALLEL_MIN_PAGES).

Measured crossover is ~160 wafers (100 wafers: parallel is 0.57×, i.e. slower; 500 wafers: 1.94× faster). With both passes parallel, a 500-wafer report improved from ~55 s to ~25 s ($\sim 2.2\times$).

6.4 Adaptive per-wafer DPI

The individual per-wafer archive PNGs use an adaptive output resolution keyed to how many threshold zones the wafer's die values span (report._zone_count, computed from the raw values — no interpolation needed):

- a wafer entirely within one zone renders as a single flat colour and is saved at WAFER_PNG_DPI_UNIFORM (70 dpi);

- a wafer spanning more than one zone has colour boundaries worth resolving and is saved at `WAFER_PNG_DPI_MULTIZONE` (150 dpi).

Measured: a uniform wafer → 560 × 560 px (∼5.6 KB); a three-zone wafer → 1200 × 1200 px (∼14 KB). The rule keys off the zone count, not the specific colour, so an all-yellow or all-red wafer is also treated as uniform. The PDF grid cells are vector (resolution-independent) and the PPTX page snapshots mix 25 wafers, so adaptive DPI applies only to the standalone per-wafer images.

6.5 Overview charts (`exporters/report_charts.py`)

Headless, Matplotlib-only renderers for the report's front-page **Threshold Scatter** and **Value Histogram**, colour-matched to the wafer maps:

- Scatter plots value against wafer (jittered into a per-wafer strip), downsampling above 200,000 points.
- Each threshold zone gets a distinct **shape** (♦ diamond below Limit 1, ■ square mid, ● circle above Limit 2).
- Marker size/alpha are **adaptive**: big and bold for sparse parameters, small and light for dense ones (100k+ dies) so they do not smear into solid bands ($\sim$ 1k pts $\rightarrow s \approx 20$; $\sim$ 200k pts $\rightarrow s \approx 3$).

Chapter 7

Exporters

7.1 PDF structure

`pdf_exporter.generate_pdf` builds a head section in the main process and merges per-lot grid pages (rendered in worker processes) with `pypdf`, attaching bookmarks. Page order:

1. **Threshold Scatter** (page 1)
2. **Value Histogram** (page 2)
3. **Summary Report** (lot/wafer counts, file metadata)
4. **8-Condition Distribution** bar chart
5. **Per-lot wafer grids** — a 5×5 grid (`MAX_WAFERS_PER_PAGE`$= 25$) per page

Each grid page is a self-contained single-page *vector* PDF produced by a picklable worker (`_render_grid_page`); the main process merges them in order, so parallelism costs no quality.

7.2 PPTX

`create_powerpoint_report` is fed the PNG snapshots that the PDF pages emit as a side effect. It sorts them by leading page number (numerically, so a 2000-slide deck past page 999 stays ordered), builds a title slide, and adds one image slide per page. Consequently the deck order matches the PDF, including the scatter and histogram front pages.

7.3 CSV

`export_color_summary_csv` writes a per-wafer colour-zone count summary.

Chapter 8

The Graphical Interface

8.1 Main window

`ui/main_window.DataProcessorUI` is a split view: a left control panel (input file, thresholds, orientation, output folder, run/automation buttons) and a right panel of tabs.

Tab / widget	**Purpose**
Live Preview (5×5)	Wafer thumbnails stream in as they render; resets per lot.
Threshold Scatter	Interactive scatter of the loaded parameter with log-Y toggle.
Histogram	`HistogramWidget` — colour-zoned distribution with log toggles and a stats bar.
Wafer Orientation	Live diagram reflecting rotation / mirror choices.

8.2 EFF loading in the GUI

Choosing a `.eff` file opens the **parameter picker** (`eff_param_dialog`): Lot/Wafer/X/Y are shown as always-loaded coordinates; the user ticks one or more measurement parameters (filter, select-all, "with spec limits" helpers). On acceptance the tool enters *EFF mode*.

8.3 Per-parameter preview pager

In EFF mode a selector bar appears above the tabs:

```
[<- Prev]  [ 1/N  <parameter>  v ]  [Next ->]   <param>: 1,475 pts . 25 wafers
```

Paging (or the dropdown) loads that parameter's data on an `EffPreviewWorker` background thread and updates the scatter + histogram. Results are cached by column index, so revisiting a parameter is instant. On navigation the tool also:

- auto-fills **Limit 1** / **Limit 2** from that parameter's EFF spec limits, and
- shows the parameter name in the chart titles and a label in the Parameters panel.

8.4 Run & worker signals

"Generate Report" dispatches to `EffReportWorker` (EFF mode) or `ReportWorker` (txt/csv). Both are `QThread`s emitting `progress`, `wafer_progress`, `status_message`, `wafer_ready`, an error signal, and a completion signal (`report_done` / `all_done`). The window connects these to the two progress bars, the live grid, and the status line, and can cancel mid-run.

Chapter 9

Automation

9.1 Saving & replaying jobs

“Save Job” gathers the current inputs into a `WaferJob` and writes a `.wtjob`. `automation/runner.py` replays it headlessly: `run_job` → `_run_loaded`, dispatching to `process_eff` for `.eff` inputs or `generate_report` for txt/csv.

9.2 Exit codes

The runner returns a distinct exit code per outcome, surfaced to Windows Task Scheduler as `LastTaskResult`:

Code	Constant	Meaning
0	`EXIT_OK`	report generated
1	`EXIT_ERROR`	unexpected error during generation
2	`EXIT_USAGE`	no job path given
3	`EXIT_NO_OUTPUT`	ran but produced no PDF
10	`EXIT_MISSING_INPUT`	the job’s input file (e.g. `.eff`) is missing
11	`EXIT_NO_MATCH_IN_FOLDER`	folder-mode job matched no file
12	`EXIT_BAD_JOB`	the `.wtjob` is missing / invalid

9.3 Windows Task Scheduler integration

`automation/scheduler.py` registers a task named `WaferTool_Job_<name>` per job. Daily and weekly triggers use PowerShell `ScheduledTasks` cmdlets; monthly uses Task Scheduler XML (PowerShell has no monthly-trigger cmdlet). Tasks run as the logged-on user with no stored password. The command line is either the frozen executable (`App.exe --run-job <job>`) or `python run_job.py <job>` in development.

9.4 Schedule Jobs dialog & Status column

`scheduler_dialog.SchedulerDialog` lists the registered tasks and lets the user schedule / run-now / remove them. A colour-coded **Status** column maps the last exit code (via `_status_for`) to a readable outcome:

Status	**Colour**	**Trigger**
✓ Success	green	code 0
× Missing input file (EFF)	red	code 10
△ No matching file in folder	amber	code 11
× Bad / missing job file	red	code 12
× Ran but no output	red	code 3
× Failed (error)	red	code 1 / other
▶ Running...	blue	Task Scheduler "running"
— Not run yet	grey	never run

Chapter 10

Concurrency Model

- **Qt threads** — long operations (report generation, EFF parameter preview) run on `QThread`s so the UI never blocks. They communicate with the UI only through Qt signals.
- **Process pools** — CPU-bound rendering (wafer PNGs, PDF grid pages) fans out across processes with `ProcessPoolExecutor`; results are consumed in submission order so the live preview stays ordered.
- **Cancellation** — a `threading.Event` in each worker; the pipeline observes it through the progress callback (raising `RuntimeError`) and at chunk / parameter boundaries.
- **Within vs. across parameters** — the EFF pipeline processes parameters sequentially, each internally parallel, to keep all cores busy without oversubscribing nested pools.

Chapter 11

Performance & Scalability

11.1 What scales well

EFF extraction *streams* (no whole-file materialisation); per-wafer grids are freed immediately after drawing; rendering is parallelised; the PPTX slide images are produced as a by-product of the PDF pages rather than a third pass; and uniform wafers are saved at a lower DPI (see § 6) so common single-colour wafers cost little disk.

11.2 Known limits

1. **No wafer-count ceiling.** Everything downstream is $O(n_{\text{wafers}})$: the statistics loop, the CSV loop, N PDF pages, N PNGs, N slides. A file that resolves to ∼195,000 wafers (dense parametric data across many lots) makes a run impractical. The recommended fix is a *guardrail* that warns before rendering an unreasonable wafer count.
2. **Pure-Python per-wafer loops** in `statistics.py` and `csv_exporter.py` iterate every group in Python; vectorising with a single pandas `groupby().agg()` would cut these dramatically at 100k+ wafers.
3. **Full-file materialisation on the txt path** — the converted-text loader reads the entire file into memory.
4. **Interpolation is cheap for sparse wafers.** A measured experiment showed that caching interpolated grids to disk to avoid a second interpolation was *slower* than recomputing (disk load ≈ 52 ms vs. `griddata` ≈ 11 ms for ∼9-point wafers), so that optimisation was deliberately *not* adopted.

Chapter 12

Modularity Assessment

Strengths. Clear downward-only layering; the EFF layer (`eff_loader` → `eff_pipeline` → `eff_worker`) cleanly separates I/O, GUI-free orchestration, and the Qt adapter; exporters are isolated behind functions; one core renders for both GUI and report.

Opportunities.

1. `ui/main_window.py` is large (∼1.6k lines) and mixes reusable canvases, the main window, and business logic (`_gather_job`, EFF wiring). Extracting the canvases to their own module and moving job-gathering out of the UI would slim it.
2. The scatter/histogram exist *twice* (GUI canvases vs. `report_charts`); sharing one axes-level renderer would remove drift.
3. Some dead / legacy code (an unused `HistogramCanvas`, `*_old.py` files) can be removed.

Chapter 13

Build & Run

- **Interactive:** `python main.py` (PyQt6).
- **Headless job:** `python run_job.py <job.wtjob>` — fixes `sys.path` then calls `automation.runner.main`.
- **Packaging:** PyInstaller spec files (`main.spec`, `WaferMapTool.spec`); the PPTX template and compiled parser are bundled and resolved from `sys._MEIPASS` when frozen.
- **Dependencies:** PyQt6, matplotlib, numpy, pandas, scipy, pypdf, python-pptx (optional; PPTX is skipped gracefully if absent).

End of document